



AEROTWIN

PROPULSION INTELLIGENCE

Technical Documentation

A residual-driven digital twin for MALE UAV aero piston engines. A physics model runs in lockstep with the engine on the same inputs; the difference between them — the residual — is what every detector downstream reads. This document describes how the prototype is built, how each layer works, and what it deliberately refuses to claim.

PROGRAMME

PS-26054

MODEL

FGPISTON-180 v1.0.0

BUILD STATUS

Research prototype

AIRWORTHINESS

Not certified

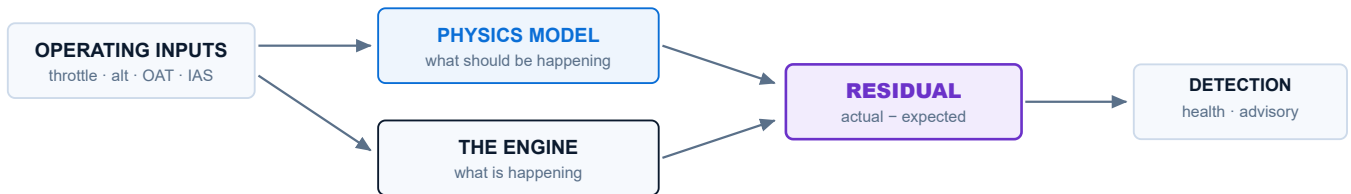
Contents

01	The Core Claim	what makes this a twin and not a dashboard
02	System Architecture	two processes, one contract
03	The Data Layer	the source contract and its four implementations
04	Provenance	measured, generated, derived — and why it is enforced
05	The Physics Model	how the expectation is produced
06	The Residual Engine	normalisation, drift, persistence
07	State Estimation	separating the engine's identity from its faults
08	Detection and Fusion	a readable baseline and a learned model
09	Abstention and the Model Envelope	when the system refuses to answer
10	Explanation and Advisory	from a score to an action
11	The Simulation Clock	one authority for everything that moves
12	The Console	routes, screens, and the 3D stage
13	API Reference	REST and WebSocket surface
14	Running the Prototype	installation and the demonstration path
15	Verification	what was tested and what the results were
16	Limitations	stated before anyone has to find them

01 The Core Claim

Most engine-monitoring systems compare a sensor reading against a threshold. That works for failures and is nearly useless for degradation, because a cylinder can run thirty degrees hotter than it should and still sit well inside its redline — the reading is *normal for the limit* and *abnormal for the conditions*.

AEROTWIN is built on the second comparison. Every second:



THE LOCKSTEP LOOP. BOTH HALVES SEE IDENTICAL INPUTS; ONLY THE RESIDUAL LEAVES.

THE INVARIANT

Nothing in the analytics stack reads a raw sensor value. The detector, the health index, the explanation and the advisory all consume residuals. That is the difference between a twin and a dashboard with a 3D model on it, and it is enforced structurally: the detector's feature vector is built from `ChannelResidual` objects, which do not expose an unnormalised reading.

Why per-cylinder asymmetry is the diagnostic basis

All four cylinders share an intake, an exhaust system, a cooling airflow and a crankshaft. Anything that changes the whole engine — a throttle movement, a climb, a hot day — moves all four together. A fault in *one* cylinder moves one of them relative to the other three. So the quantity that carries fault information is not CHT, and not even the CHT residual, but the **dispersion of the four CHT residuals about their own median**. This is why the system can localise to a cylinder at all, and why a corpus without per-cylinder instrumentation cannot support that claim (§09).

02 System Architecture

Two processes. The backend owns the twin and the truth; the frontend is a viewer with one exception — it can run a reduced model of its own when the backend is absent, and says so on every value.

LAYER	TECHNOLOGY	RESPONSIBILITY
Twin runtime	Python 3, asyncio	The lockstep clock at 5 Hz, stepping 1 Hz samples through physics, residual, estimator and detector
Physics	In-process FGPiston-class model; JSBSim optional	Produces the expectation for every channel from the operating inputs
Analytics	NumPy, SciPy, scikit-learn	Residual engine, state estimator, baseline GLM, Isolation Forest, calibration, advisory
API	FastAPI	REST surface and four WebSocket topics
Console	React 18, TypeScript, Zustand	Thirteen operator screens and the simulation control surface
3D	Three.js, React Three Fiber	Aircraft, sector, terrain, engine assembly, thermal and airflow views
Charts	Apache ECharts	Expected/actual pairs, residual bands, health trend

The lockstep loop

The backend's clock runs at 5 Hz and advances `time_scale ÷ 5` one-second samples per tick, so a 20× replay steps four samples per tick and a 200× demonstration steps forty. The loop is wrapped so a failure anywhere in the analytics stack degrades the diagnosis rather than stopping the stream — a ground station that goes silent is a worse failure than one reporting reduced capability.

```
while running:
    if not paused and datalink:
        for _ in range(samples_this_tick):
            frame = source.read()          # 1 Hz sample, any source
            runtime.step(frame)            # physics -> residual -> estimator -> detector
        broadcast()                        # four topics, only what changed
        sleep(0.2 - elapsed)
```

03 The Data Layer

The architectural centrepiece is `sources/base.py`. Every origin — a real corpus, the team corpus, the demonstration simulator, a live CAN bus — implements the same interface and emits the same frame shape. Everything downstream is written against that contract and never against a particular source.

```

class DataSource(abc.ABC):
    @property
    def descriptor(self) -> SourceDescriptor: ...
    def open(self, **kwargs) -> None: ...
    def read(self) -> TelemetryFrame | None: ...

@dataclass
class TelemetryFrame:
    t: float                # mission elapsed seconds
    wall_clock: str
    flight_id: str
    source_id: str
    phase: str
    channels: dict[str, float] # sensor readings
    inputs: dict[str, float]  # control + ambient, shared with the twin
    provenance: dict[str, str] # per channel: REAL | SIMULATED | DERIVED

```

WHY THIS MATTERS

Adapting AEROTWIN to a DRDO/VRDE test rig is **one class**. Not a rewrite, not a re-training, not a schema migration — one implementation of three methods. The claim is testable, and §03.2 is the test: a corpus with a different schema, different units and a different level of instrumentation was dropped in without touching anything downstream.

03.1 The four sources

SOURCE	KIND	PROVENANCE	NOTES
NGAFID replay	REPLAY	REAL	NGAFID-MC (Zenodo 6624956, CC-BY-4.0), mounted via <code>AEROTWIN_NGAFID_DIR</code> , never vendored — 5.4 GB of 1 Hz flight data
Team corpus	REPLAY	SIMULATED	4,200 rows shipped in-repo; see below
Demo simulator	SYNTHETIC	DEMO	Runs the same thermodynamics against a <i>hidden</i> engine state the twin cannot see
SocketCAN	LIVE	REAL	Deployment interface for a real engine bus; implemented, not exercised in this build

The demonstration simulator deserves a note. It is **not** a random number generator. It runs the same thermodynamic model the twin uses, but against a hidden truth carrying a unit-specific fingerprint (volumetric efficiency, cooling effectiveness and per-cylinder trim slightly off nominal — every real engine has one), a progressive degradation mechanism, thermal inertia, sensor noise and slow sensor bias. The twin is never told about any of it. The residual it computes is therefore a genuine emergent quantity rather than a scripted number.

03.2 The team corpus, and what it proved

The corpus from the team's data workstream (`MONISHA-tech316/aerotwin-person1`) is generated by a documented simulator — its own generator adds Gaussian noise to fixed baselines — so every frame is tagged `SIMULATED` and its `state` / `fault` columns are described as generator labels, never as maintenance findings.

FILE	ROWS	RATE	GROUND TRUTH
normal.csv	2,000	1 Hz	NONE
degradation.csv	1,000	1 Hz	MECHANICAL_DEGRADATION
faults.csv	1,200	1 Hz	OVERHEATING, VIBRATION, PRESSURE_DROP, MECHANICAL_DEGRADATION (300 each)

UNIT DECISIONS, RECORDED RATHER THAN HIDDEN

Each mapping is a judgement about what the generator's baseline physically *is*. Every one is stored with its source column and factor, and the Dataset screen renders the conversion table so it can be checked:

SOURCE	UNIT	CHANNEL	UNIT	REASONING
pressure	bar	oil_press_psi	psi	×14.5038 — 4.2 bar → 61 psi, nominal oil pressure for this engine class
temperature	°C	oil_temp_c	°C	Head temps run 150–250 °C; a 78 °C baseline is an oil temperature, and OVERHEATING taking it to 105–125 °C fits that reading
fuel_flow	kg/h	fuel_flow_gph	gph	÷2.7255 — avgas at 0.72 kg/L
altitude	m	altitude_ft	ft	×3.28084
rpm, vibration, ambient_temperature	—	rpm, vibration_g, oat_c	—	Direct

The original row is preserved on every frame, so nothing is lossy and the conversion can always be checked against the source. Eleven AEROTWIN channels have no column in this corpus — eight per-cylinder, plus manifold pressure, airspeed and injection timing — and the adapter reports them as absent rather than synthesising them.

04 Provenance

Eighteen channels at 1 Hz. Sixteen are measurable from a corpus; two have no counterpart in any available corpus and are produced by the physics model.

CHANNEL	UNIT	SOURCE	CHANNEL	UNIT	SOURCE
RPM	rpm	Measurable	Oil pressure	psi	Measurable
Manifold pressure	inHg	Measurable	Oil temperature	°C	Measurable
Fuel flow	gph	Measurable	OAT / Altitude / IAS	°C, ft, kt	Measurable
EGT 1–4	°C	Measurable	Vibration	g rms	SIMULATED
CHT 1–4	°C	Measurable	Injection timing	°BTDC	SIMULATED

A DISTINCTION THE CODE ENFORCES

A channel's registry provenance says whether it *can* be measured — that is a property of the contract. Whether it *was* depends on which source currently holds the live seat, and only the source descriptor knows that. Every **REAL** badge in the console passes through a single selector that consults the descriptor, so a badge can never end up on the output of the simulator. Two different questions, two different answers, one gate.

05 The Physics Model

A lumped-parameter FGPiston-class model of a 180 hp, four-cylinder, horizontally opposed, air-cooled, fuel-injected engine (Lycoming IO-360 class). It consumes throttle, mixture, altitude, OAT and airspeed and produces an expectation for every modelled channel.

STAGE	WHAT IT COMPUTES
Atmosphere	Density ratio from pressure altitude via the ISA relation $(1 - 6.875e-6 \cdot h)^{4.2561}$
Induction	Manifold pressure from throttle position and ambient density; volumetric efficiency from RPM and MAP
Combustion	Indicated power from air mass flow and mixture; brake power after friction; fuel flow from BSFC
Thermal	Head heat rejection against cooling mass flow — a function of airspeed and density — giving CHT; exhaust enthalpy giving EGT
Lubrication	Oil pressure from RPM; oil temperature from power and OAT
Inertia	First-order lag applied identically to expectation and observation ($\tau \approx 34$ s CHT, 7 s EGT, 150 s oil)

WHY THE LAG IS APPLIED TO BOTH SIDES

Cylinder heads and oil do not follow a power change instantly. If only the observation carried thermal inertia, every throttle transient would manufacture a residual that decayed over the next half-minute and the detector would fire on ordinary flying. Filtering both halves with the same time constants means a transient produces no residual at all — which is correct, because nothing is wrong.

Installing `jsbsim` switches the physics half from the in-process model to JSBSim itself, with no other code change. The physics backend reports which one is active on the Architecture screen.

06 The Residual Engine

A raw difference is not comparable across channels: eight degrees of CHT and eight rpm are not the same size of surprise. Each channel therefore carries a **scale** — the standard deviation of its residual over observed healthy, steady-state operation — frozen once the baseline locks.

QUANTITY	SYMBOL	MEANING
Residual	$r = \text{observed} - \text{expected}$	The raw difference, in the channel's own unit
Normalised	$z = r / \text{scale}$	Comparable across channels; the detector's input
Drift	$\text{EWMA}(r)$	Slow component — a developing fault, not a gust
Persistence	$\text{CUSUM}(z - k)$	How long the deviation has held; separates a real drift from a spike
Asymmetry	$\text{EWMA}(r)_i - \text{median}$	Per-cylinder deviation from its three siblings — the localising quantity

The steady-state gate matters. Residuals are only admitted to the baseline and to the learned model while the operating point is steady, because a transient residual is a modelling artefact and training on it teaches the detector that artefacts are normal.

07 State Estimation

Every engine leaves the factory slightly different, and those differences are not faults. The state estimator separates the two by fitting three slowly varying parameters that describe *this* engine's identity:

- **Volumetric efficiency** — how well this engine breathes relative to nominal.
- **Cooling effectiveness** — how well its cowling and baffles reject heat.
- **Per-cylinder trim** — the fuel/air distribution across the four cylinders.

These are estimated from healthy operation and then held. A fault appears as a residual the estimator *cannot* absorb, which is exactly the definition wanted: a persistent, localised, unexplainable deviation. The estimator is gated — it reports **LEARNING** until it has enough steady healthy evidence and only then **LOCKED**, and while it is learning the detector's confidence is capped.

08 Detection and Fusion

Two detectors, deliberately different in character, run on the same residual features.

08.1 The transparent baseline

A linear model over named residual features. It is not there because it is the most powerful thing available — it is there because a propulsion engineer can read it, argue with it, and change a coefficient. Every term has a physical meaning and a unit. When the system says a cylinder is deviating, this is the half that can be interrogated.

08.2 The learned model

Algorithm	Isolation Forest, 200 estimators (scikit-learn)
Supervision	Unsupervised — fitted on normality, not on labelled faults
Input	9 residual features: CHT/EGT asymmetry, CHT drift, normalised z, persistence, trim divergence, EGT trim divergence, CHT/EGT agreement, CHT dispersion
Training set	240 steady-state healthy residual vectors pooled across four cylinders, of <i>this</i> engine
Until then	Reports UNTRAINED and contributes exactly zero to the fused score

Fitting on this engine's own healthy residuals — rather than on a fleet — means the model learns what normal looks like *here*, so a unit that has always run one cylinder slightly warm does not generate a permanent false positive.

08.3 Fusion

The physics baseline keeps the majority weight and an abstaining learned model changes nothing at all, so the fused score can never be worse-founded than the transparent half alone. A learned model that disagrees can raise or lower confidence; it cannot manufacture a detection on its own.

09 Abstention and the Model Envelope

The system is designed to be able to say it does not know. There are two independent abstention paths.

09.1 Insufficient evidence

Where the residual is present but thin — not persistent enough, not seen across enough regimes, estimator not yet locked — the anomaly is returned with `abstained: true` and a stated reason, and the console renders **MODEL ABSTENTION** rather than a diagnosis. A mechanism is always named as `LIKELY CONTRIBUTING`, never as confirmed.

09.2 Outside the model envelope

This one was added because the team corpus exposed it. The expectation comes from a model calibrated for a 2,700 rpm, four-cylinder air-cooled aero engine. Hand it a corpus recorded from a different machine and every residual is enormous — not because anything is wrong with that engine, but because the model was not built for it. The health index collapsed to zero and the engine was reported CRITICAL.

THE FAILURE THIS PREVENTS

That number would have been a lie. The residual was real, but it was a *model mismatch* residual, not evidence of a fault: nothing had been observed about that engine except that AEROTWIN's model was not built for it. Reporting a failing engine from that is precisely the failure mode this system exists to be immune to.

So the envelope is checked before anything is diagnosed:

VERDICT	CONDITION	BEHAVIOUR
IN ENVELOPE	RPM within calibration, full channel coverage	Diagnose normally
REDUCED	Model applies, instrumentation incomplete	Diagnose what is covered; name what is not
OUT OF ENVELOPE	RPM beyond 1.15× rated, or coverage under 30%	Abstain — no health verdict, no anomalies, no advisory

On abstention `health_index` is returned as `null`, not `0`, and the gauge draws a dash rather than a zero. Absolute threshold alerts still fire: a redline is a redline whatever model is running.

Replaying the team corpus is a live demonstration of the whole mechanism:

```
MODEL ABSTENTION
Crankshaft speed 4,726 rpm is above the model's calibrated ceiling of
3,105 rpm (Lycoming IO-360 class, rated 2,700 rpm). This is a different
class of engine.
No per-cylinder instrumentation in this source: cylinder-level
localisation is unavailable. Bulk channels only.

coverage      30.8% (4 of 13 residual channels)
localisation   unavailable
health_index   null
```

10 Explanation and Advisory

A score is not an answer. Every anomaly carries the evidence that produced it, so the operator sees the reasoning rather than a number:

FIELD	CONTENT
Affected channel & cylinder	Which signal, on which cylinder
Actual / Expected / Residual	The three numbers, in the channel's own unit
Trend & persistence	Drifting or stable; how many samples it has held
Regime coverage	How many distinct power/altitude regimes it has been seen in
Feature contributions	Which residual features drove the score, and by how much
Mechanism	Named as LIKELY CONTRIBUTING — e.g. early exhaust valve distress
Confidence	Calibrated, or ABSTAINED with the reason

The advisory layer turns that into an action — an inspection recommendation scoped to a cylinder, with the serviceability outlook from the health trend. It is an advisory, not a maintenance authorisation, and the console says so.

11 The Simulation Clock

Everything that moves because the *simulation* is moving derives from one object. Nothing else in the console owns a timer that advances simulation state. That single rule replaced an architecture in which the console had a second clock of its own — a 220 ms interval driving the in-browser model, which knew nothing about the paused flag and kept stepping through a pause.

STATE	MEANING
idle	Nothing loaded, nothing running
starting	Loading data, initialising twin and detector — not yet advancing; each stage named on screen
running	The clock is advancing
paused	Held; index and time do not change
stopping / stopped	Torn down, restartable; computed state kept for inspection
completed	The source was exhausted; holds on the last real sample
error	Start-up or transport failed; the reason is on screen

11.1 Two transports, one state machine

LIVE commands the backend and follows the frames it sends. **LOCAL** drives the reduced in-browser model when no backend answers. The state machine, the controls and every panel behave identically; only the transport differs. This is what makes the full control surface work on a static deployment, where it previously did nothing at all.

11.2 What PAUSE holds

Pause is not a UI state. When it is pressed, the clock stops and with it: the dataset index, telemetry, the charts, the residual, the twin, anomaly processing, health, mission progress, the UAV's flight integration, the contrail, the piston

and crankshaft animation, the cooling-airflow particles, and the narration timeline. Decorative hover motion continues; nothing simulation-driven does.

Two subtleties are handled explicitly. A live frame already in flight when the command went out is **dropped at the gate**, so no number moves a beat after the press. And because the console pauses optimistically — held on the press, not on the round trip — it then **settles onto the sample the backend actually stopped on**, which the pause response reports. The index on screen is the index the twin is holding at.

11.3 The control surface

CONTROL	BEHAVIOUR
START	Dataset → simulation → twin → AI → telemetry, each stage named. Pressing it twice never creates a second loop
PAUSE	As §11.2
RESUME	Continues from exactly that timestep — never restarts, never skips, never re-prepares the source
STEP	Advances exactly one 1 Hz sample regardless of replay rate, then holds again
STOP	Ends the sortie; everything computed stays inspectable
RESET	Rewinds and clears every derived quantity — no residual baseline or CUSUM carried into the next run
1× ... 200×	Replay rate. Changing it never resumes a held simulation and never spawns a second loop

The offered buttons are *derived* from the status rather than written out by hand, so PAUSE is never offered on an already-held simulation and STEP is never offered on a running one.

AUTHORITY

In LIVE mode the backend owns the clock, so it is the authority on whether anything is advancing. The status poll reconciles: a console showing a running simulation over a server paused from somewhere else is the same lie as a pause that does not pause, in the other direction.

12 The Console

ROUTE	PURPOSE
/intro	Cinematic first-entry sequence — skippable, replayable, remembered in <code>localStorage</code>
/uav	Interactive 3D aircraft; engine inspection with PHYSICAL / THERMAL / AIRFLOW / TWIN views
/dashboard	Command Center — health, twin sync, telemetry, the pipeline
/twin	Observed → physics → expected → residual → state → updated twin
/engine	Per-cylinder health, thermal map, expected-vs-actual table
/telemetry	All channels with provenance badges and expected/actual pairs
/anomalies	Ranked anomalies and the evidence behind each
/prognostics	Health trend and serviceability outlook
/maintenance	Advisories, scoped to a cylinder
/simulation	Mission profile simulation over configurable sorties
/mission	Sector picture, route, mission state
/dataset	The mounted corpus, counted: files, rows, validity, rate, labels, mapping, gaps
/data	Data dictionary, ingest report, model cards
/validation	Validation harness, calibration, limitations, domain gap
/architecture	Live system architecture with per-node state

The 3D stage

One never-unmounted canvas serves the corner card, the Command Center hero and the fullscreen mission view; it animates between those framings rather than remounting, so the aircraft is continuous across the transition. The engine assembly is a real four-cylinder layout — crankcase, four barrels with cooling fins, pistons, connecting rods, crankshaft, propeller flange — and its **crank angle is integrated from RPM**, so the pistons turn because the simulation is turning them. When the simulation holds, so do they.

If WebGL is unavailable the stage falls back to a 2D attitude view driven by the same flight model, so the same numbers are shown with a context every machine has. An error boundary guarantees a lost GL context degrades the visualisation rather than blanking the telemetry.

The narrated demonstration

`RUN STORY` plays the whole chain full screen over the aircraft in about ninety seconds — link, telemetry, twin sync, healthy engine, first deviation, residual growth, cylinder separation, estimator, detector, explanation, advisory. The narration is scripted; every number it shows is read live from the store at that moment, so it cannot state something the system is not actually reporting. It reads the sortie already in progress rather than rewinding it, and its pause control pauses the simulation it is narrating.

13 API Reference

WEBSOCKET TOPICS

TOPIC	PAYLOAD
/ws/telemetry	Tick, channels, expected, residuals, inputs, engine summary, datalink, source descriptor
/ws/residuals	Per-channel residual, normalised, EWMA, CUSUM, scale; estimator state; asymmetry
/ws/alerts	Ranked anomalies with contributions, advisories, threshold alerts, event log
/ws/mission	Mission state, position, altitude, IAS, RPM, power, replay rate

REST — STATE

GET /api/system/status	Boot, datalink, twin, model, running/paused, replay rate, uptime
GET /api/system/architecture	Live architecture graph with per-node state
GET /api/engine/status	Health index (or <code>null</code> on abstention), envelope, cylinders, prognosis
GET /api/engine/comparison	Expected vs actual, per channel and per cylinder
GET /api/residuals	Current residual frame
GET /api/twin/state	Estimator parameters and lock state
GET /api/anomalies	Ranked anomalies
GET /api/anomalies/{id}/explain	Full evidence for one anomaly
GET /api/validation	Harness metrics, calibration, limitations, domain gap
GET /api/ml/status	Model cards for both detectors

REST — CONTROL

POST /api/replay/pause	Hold; returns the exact sample it stopped on
POST /api/replay/resume	Release; returns the sample it resumed from
POST /api/replay/step	Advance exactly N 1 Hz samples while held
POST /api/replay/stop	Hold and mark stopped; derived state kept
POST /api/replay/reset	Cold restart: rewind source, clear every derived quantity
POST /api/replay/speed	Set replay rate
POST /api/demo/start · /stop	The compressed demonstration sortie
POST /api/system/datalink	Simulate a link interruption — the twin holds rather than guessing

REST — DATASET

GET /api/dataset/status	Corpus counted off the filesystem on every call
GET /api/dataset/validate	Per-file PASS / WARNING / ERROR with the reason
POST /api/dataset/activate	Put a corpus file on the live seat
POST /api/dataset/restore	Return the seat to the demonstration simulator

14 Running the Prototype

```
git clone https://github.com/shanmugapriya180607/AeroTwin-AI.git
cd AeroTwin-AI

# backend
cd backend && pip install -r requirements.txt

# frontend
cd ../frontend && npm install
```

One command starts both and opens the console:

```
./start.ps1      # Windows
./start.sh       # macOS / Linux
```

Console <http://localhost:5173/>

API documentation <http://127.0.0.1:8011/docs>

THE DEMONSTRATION PATH

1. Open the console — the cinematic intro plays on first entry, skippable and replayable.
2. Inspect the aircraft and the engine on `/uav` : physical, thermal, airflow, twin.
3. Enter the Command Center. The sortie already in progress is adopted and shown.
4. Press **RUN STORY** for the ninety-second narrated chain, or drive it manually.
5. Watch the residual grow on `/twin` ; watch cylinder 3 separate on `/engine` .
6. Press **PAUSE**. Every number holds. Press **STEP** to walk it one second at a time.
7. Open `/anomalies` for the evidence, `/maintenance` for the advisory.
8. Open `/dataset` and press **REPLAY** on a team-corpus file to watch the twin abstain.

DEPLOYMENT WITHOUT A BACKEND

The console stays fully demonstrable as a static bundle: the local transport takes the seat, `DEMO` appears on every value, and the complete control surface works. The Dataset screen reads a report generated at build time by the same code the API serves — identical figures, marked `BUILD SNAPSHOT` rather than passed off as a live inspection.

```
cd backend && python -m tools.export_dataset_report
```

15 Verification

15.1 Detector performance

Against the synthetic harness — 48 trials, 24 positive and 24 negative, at a 0.50 operating threshold:

METRIC	VALUE	MEANING
ROC-AUC	0.941	Ranking quality of the anomaly score against the injected label
Precision	0.913	At the operating threshold
Recall	0.875	At the operating threshold

WHAT THESE FIGURES ARE NOT

They are measured against *injected* degradation in the demonstration simulator, not against labelled maintenance events on real hardware. They demonstrate that the pipeline separates injected faults from healthy operation. They are not a claim about field performance, and the Validation screen states this in the same words.

15.2 Simulation-engine invariants

27 assertions run against a fake scheduler, so the properties are exact rather than dependent on wall-clock timing. Among them:

INVARIANT	ASSERTION
One loop, always	START twice, RESUME twice, START-after-STOP, and speed changes each leave exactly one interval live
PAUSE holds	Index, time and clock unchanged across 100 ticks; the transport is not called; no advance listener fires
Late frames dropped	A live frame arriving after the pause command is refused and moves nothing
RESUME continues	Same timestep, then exactly one tick forward; the source is not re-prepared
STEP is exactly one	+1 s, status still <code>paused</code> ; a no-op while running
Settle after pause	The held index becomes the sample the backend actually stopped on
Adopt honestly	A backend already paused is adopted as paused, never as running
End of source	Holds on the last real sample rather than wrapping silently

15.3 End-to-end

31 checks driven through a real Chromium against the running stack — both with the backend and with it deliberately stopped. All pass. Representative measured results:

```

PASS simulation adopted the live stream      status=running transport=LIVE
PASS telemetry advances                      t 375 -> 451
PASS values EXACTLY unchanged after 6 s held t 459->459 rpm 2556.08->2556.08
PASS STEP advances exactly one timestep      t 460 -> 461, status paused
PASS RESUME continues from the same timestep t 461 -> 532
PASS STOP stops the simulation               status=stopped
PASS only one simulation loop runs           128.0 sim-s per real s
PASS engine animation held while paused      simClock.running=false
PASS dataset page shows counted figures      ROWS 4,200 VALID 4,200
PASS every route renders
PASS no unexpected console errors
=== 31/31 passed ===

```

16 Limitations

Stated here rather than left to be discovered.

AREA	STATE	DETAIL
Airframe	GAP	The available real corpus is general-aviation (Cessna 172 class), not a MALE UAV. Same engine class — four-cylinder horizontally opposed, ~180 hp, air-cooled, fuel-injected — so the thermodynamics, failure modes and sensor channels transfer. Duty cycle and airframe do not.
Vibration	SIMULATED	Vibration and injection timing have no counterpart in any available corpus and are produced by the physics model. Tagged SIMULATED everywhere they appear.
Operational data	GAP	No proprietary, operational or classified data is used or included. The repository contains none.
Physics model	SIMPLIFIED	Lumped-parameter, not CFD or FEA. Calibrated for one engine class; outside that envelope the system abstains rather than extrapolating (§09.2).
Team corpus	GENERATED	Produced by a documented simulator, not measured. Bulk channels only — no per-cylinder instrumentation, so it cannot support cylinder-level localisation.
Validation	SYNTHETIC	Metrics come from injected degradation, not labelled maintenance events on real hardware.
Live flight	OUT OF SCOPE	The SocketCAN deployment path is implemented as an interface; it is not exercised against a real engine bus in this build.

STANDING COMMITMENT

Simulated data is never presented as measured, anywhere. Where evidence is thin the system reports abstention and the reason rather than guessing, and it names a likely contributing mechanism, never a confirmed failure. A single instance of a generated channel being mistaken for a measurement would discredit everything else in the project, which is why provenance is enforced in the type system and not by convention.

AEROTWIN — Propulsion Intelligence. Programme reference PS-26054. Research prototype: **not certified for airworthiness decisions.** Sector, mission and aircraft identifiers are fictional and positions are synthetic. All components run locally; no external service is contacted at runtime.